

BIB Week 3 — Reading code you did not write

- Yazmadığınız kodu okumak
- You are not going to learn to write Python this semester. You are going to learn to read it.
- Because that is what this course grades, and that is what the job now is.
- Everything today is under ten lines. All of it is biological.

```
def count_purines(seq):  
    n = 0  
    for i in range(1, len(seq)):  
        if seq[i] == "A" or seq[i] == "G":  
            n = n + 1  
    return n
```

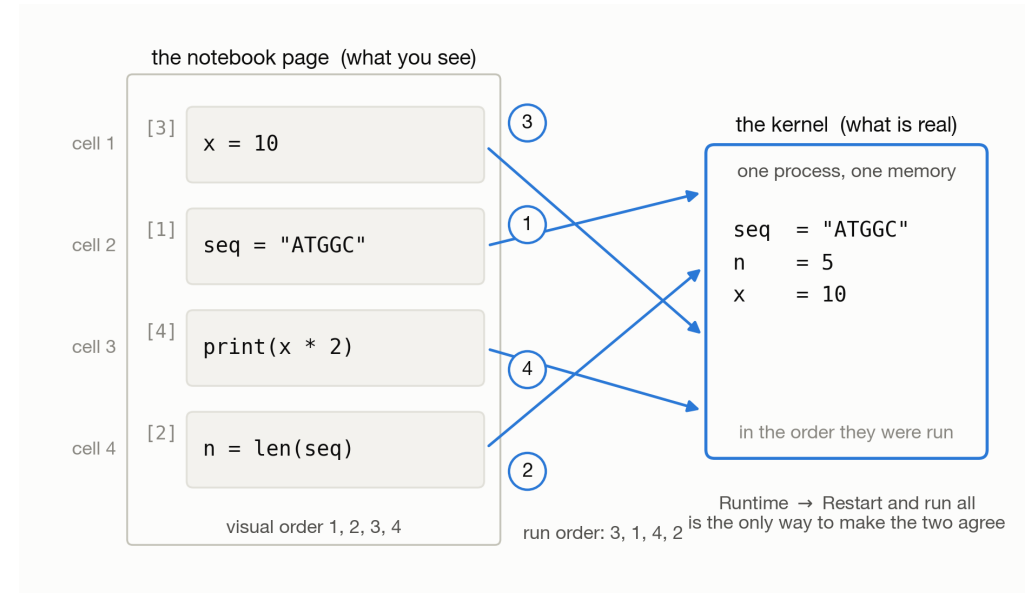
?

Ad of the Week #3

- This week: an advert that says "able to read and modify existing scripts"
- They all say it. Almost none say "able to write software from scratch."
- Tick your Skills Ledger

The Colab mental model

- A notebook (defter) is a list of cells (hücre)
- A kernel (çekirdek) is one running process with one memory
- The bracket numbers [1] [2] [7] are execution order, not position on the page
- Your notebook's state depends on what you ran, in what order — not on what you can see
- The reflex: Runtime → Restart and run all

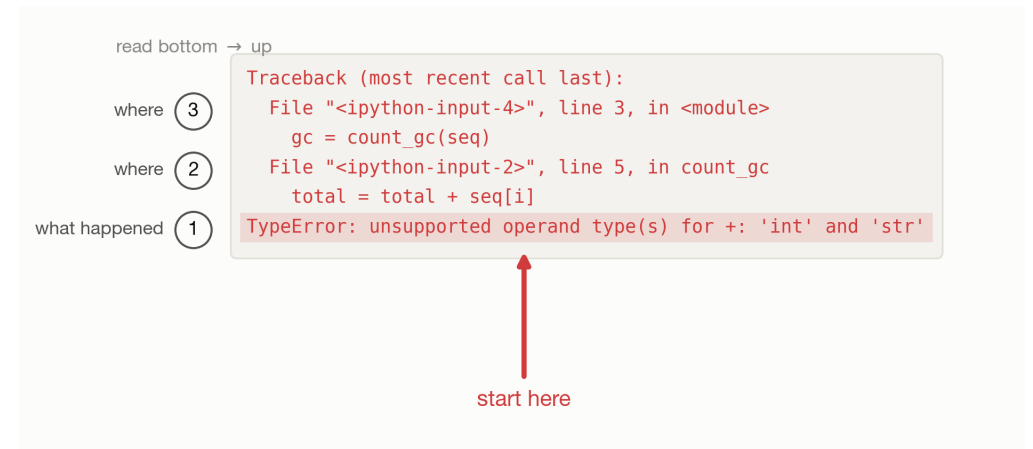


What you see is not what ran: bracket numbers are execution order.

Reading an error

- Read a traceback (hata izi) from the bottom up
- Last line = what happened. Everything above = where.
- `TypeError: ... 'int' and 'str'` — you added a number to a letter
- The red wall of text is not an insult. It is the most helpful thing Python does.

```
Traceback (most recent call last):  
  File "<ipython-input-4>", line 3, in <module>  
    gc = count_gc(seq)  
  File "<ipython-input-2>", line 5, in count_gc  
    total = total + seq[i]  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```



Read it from the bottom up.

Six constructs — the whole of today's Python

- 1 · Variable and assignment
- 2 · List and indexing
- 3 · Dictionary as a lookup
- 4 · for — iteration
- 5 · if — a branch
- 6 · def — a named block with inputs and one output
- Every example is three lines long and biological
- You will not memorise syntax. You will learn to recognise shapes.

1 variable

2 list

3 dict

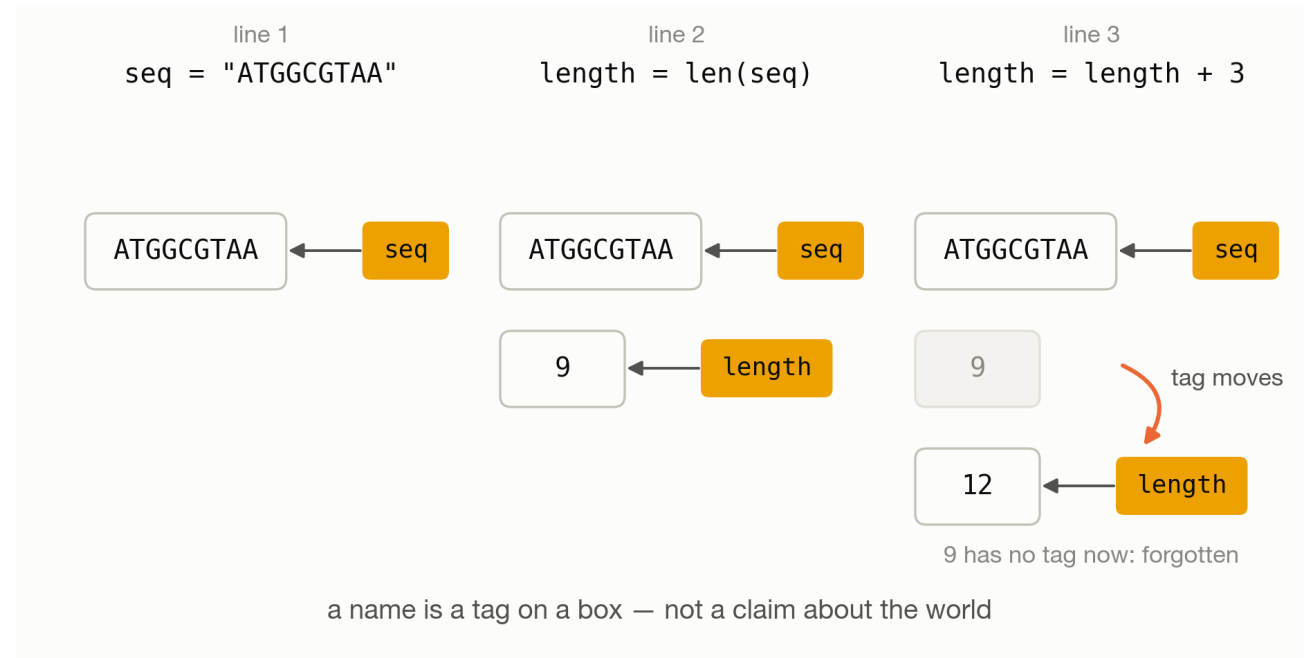
4 for

5 if

6 def

1 · Variable — a binding, not an equation

- `=` does not mean "is equal to"
- It means: evaluate the right side, then attach that value to this name
- `length = length + 3` is nonsense in algebra and routine in Python
- A variable (değişken) is a label on a box, not a claim about the world

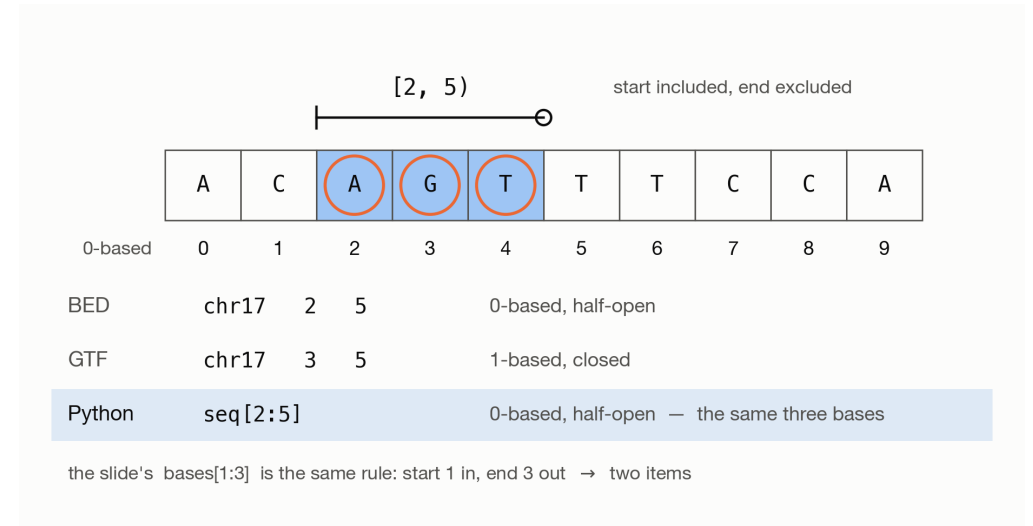


A name is a tag on a box, not a claim about the world.

```
seq = "ATGGCGTAA"  
length = len(seq)  
length = length + 3
```

2 · List — and why the first one is number 0

- A list (liste) is an ordered box of things
- Indexing starts at 0. The last index of a 5-item list is 4.
- A slice [1:3] is 0-based, half-open — start included, end excluded
- This is the same fact as last week's BED column. Not a second fact. The same one.



Same three bases, [2, 5): BED, GTF, and a Python slice.

```
bases = ["A", "T", "G", "G", "C"]
print(bases[0])    # A
print(bases[4])    # C
print(bases[1:3])  # ['T', 'G'] - two items, not three
```

3 · Dictionary — a lookup table

- A dictionary (sözlük) maps a key to a value
- Keys are things you look up by; lists are things you look up by position
- The codon table is a dictionary. So is a gene-symbol → Ensembl-ID mapping.
- A missing key raises KeyError. Loud. Good.

```
codon_table = {"ATG": "M", "TAA": "*", "GCG": "A"}  
print(codon_table["ATG"])    # M  
print("TGA" in codon_table) # False
```

Translate this to a protein using your full codon table:
ATGGCCCTGCCC

		second base			
		T	C	A	G
first base T	T	TTT F	TCT S	TAT Y	TGT C
	T	TTC F	TCC S	TAC Y	TGC C
	T	TTA L	TCA S	TAA *	TGA *
	T	TTG L	TCG S	TAG *	TGG W
C	T	CTT L	CCT P	CAT H	CGT R
	T	CTC L	CCC P	CAC H	CGC R
	T	CTA L	CCA P	CAA Q	CGA R
	T	CTG L	CCG P	CAG Q	CGG R
A	T	ATT I	ACT T	AAT N	AGT S
	T	ATC I	ACC T	AAC N	AGC S
	T	ATA I	ACA T	AAA K	AGA R
	T	ATG M	ACG T	AAG K	AGG R
G	T	GTT V	GCT A	GAT D	GGT G
	T	GTC V	GCC A	GAC D	GGC G
	T	GTA V	GCA A	GAA E	GGA G
	T	GTG V	GCG A	GAG E	GGG G

the same object, in Python

```
codon_table = {  
    "ATG": "M",  
    "TAA": "*",  
    "GCG": "A",  
}
```

key → value
look up by name, not by position

the genetic code: 64 keys → 21 values (* = stop)

The codon table is a dictionary: 64 keys, 21 values.

4 · for — do this to each one

- A loop (döngü): take each item in turn, do the indented block, move on
- The indentation is not decoration. It is the syntax. It defines what is inside the loop.
- base is a name you invented. Python does not know it means a nucleotide.
- Read it as: "for each base in this sequence..."

```
count = 0
for base in "ATGGCGTAA":
    if base == "G":
        count = count + 1
print(count)    # 3
```

```
pass 1
for base in "ATGGCGTAA":
    if base == "G":
        count = count + 1

base = "A"
base == "G" → False
count = 0
```

```
pass 2
for base in "ATGGCGTAA":
    if base == "G":
        count = count + 1

base = "T"
base == "G" → False
count = 0
```

```
pass 3
for base in "ATGGCGTAA":
    if base == "G":
        count = count + 1

base = "G"
base == "G" → True
count = 1
```

```
pass 4
for base in "ATGGCGTAA":
    if base == "G":
        count = count + 1

base = "G"
base == "G" → True
count = 2
```

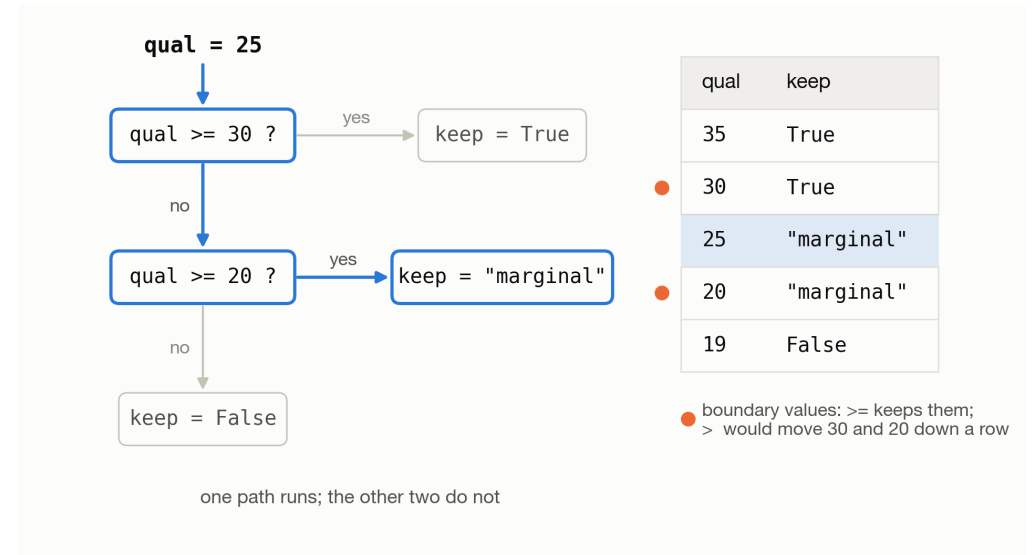
... five more passes (C G T A A) → print(count) prints 3

Four passes of one loop: base, condition, count.

5 · if — a branch

- A branch (koşul dalı): one of these blocks runs. The others do not.
- == compares. = assigns. This is a real and frequent bug.
- Only the first matching branch runs — order matters
- >= versus > is exactly one Phred point of reads. That is a design decision, not a typo.

```
if qual >= 30:  
    keep = True  
elif qual >= 20:  
    keep = "marginal"  
else:  
    keep = False
```



qual = 25 takes exactly one path; 30 and 20 sit on the boundaries.

6 · def — a named block

- A function (fonksiyon): a named block with inputs and, usually, one output
- `def name(argument):` — everything indented below is the body
- `return` hands one value back to whoever called it and stops the function
- Reading a function: look at the signature, then the return, then the middle
- The name is a claim the author is making. Check it.

```
def gc_fraction(seq):  
    g = seq.count("G")  
    c = seq.count("C")  
    return (g + c) / len(seq)
```

the name: a promise. Check it.

①

③

②

```
def gc_fraction(seq):  
    g = seq.count("G")  
    c = seq.count("C")  
    return (g + c) / len(seq)
```

signature: what goes in

body: how

return: what comes out

read the edges first: 1 signature, 2 return, 3 the middle

Read the edges first: signature, return, then the middle.

Tracing: the value table

- Input: "ATGGC"
- One column per variable. One row per pass. Fill it in.

pass	base	condition	n after
start	—	—	0
1	A	True	1
2	?	?	?

```
def count_purines(seq):
    n = 0
    for base in seq:
        if base == "A" or base == "G":
            n = n + 1
    return n
```

```
def count_purines(seq):
    n = 0
    for base in seq:
        if base == "A" or base == "G":
            n = n + 1
    return n
```

input: "ATGGC"

one column per variable
one row per pass

pass	base	condition	n after
start	—	—	0
1	A	True	1
2			
3			
4			
5			

fill it in with a pencil — that is tracing

Tracing: one column per variable, one row per pass.

Purpose, not mechanism

- Mechanism: "It loops over the sequence and adds one to a counter when the base is A or G."
- Purpose: "It counts purines."
- Both are true. Only one is useful.
- Mechanism = you read the lines.
Purpose = you understood the point.
- Every function in this course gets a one-sentence purpose statement

```
def count_purines(seq):  
    n = 0  
    for base in seq:  
        if base == "A" or base == "G":  
            n = n + 1  
    return n
```

mechanism

It loops over the sequence and adds one to a counter when the base is A or G.

purpose

It counts purines.

both are true — only one tells you whether it belongs in your pipeline

Both are true. Only one is useful.

Three kinds of defect

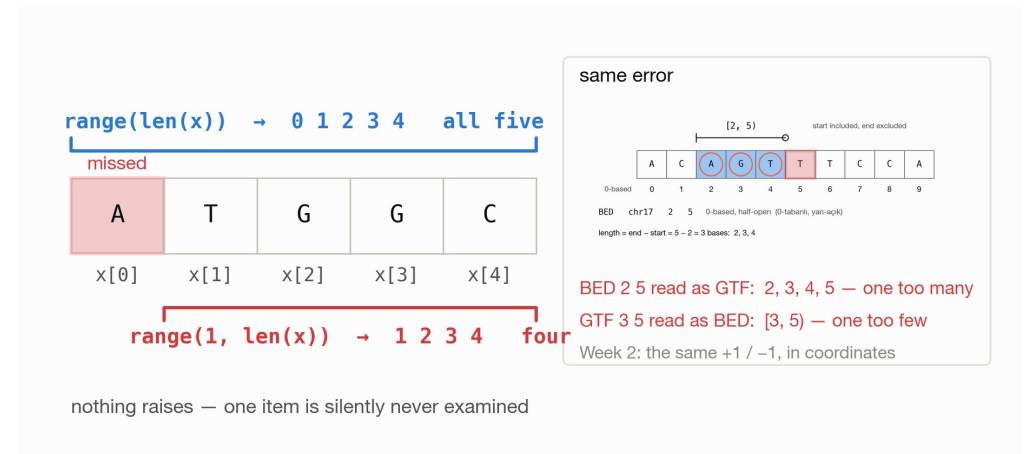
- Syntax error — the interpreter refuses to run it. You find out immediately.
- Runtime error — it starts, then stops, and prints a traceback. You find out immediately.
- Semantic defect (anlamsal kusur) — it runs, it finishes, it returns a number. It is wrong.
- The first two are free. Python finds them for you.
- The third is the entire subject of this course.

syntax error	runtime error	semantic defect
<pre>for base in seq n = n + 1</pre>	<pre>codon_table["TGA"]</pre>	<pre>count_purines("ATGGC")</pre>
<pre>File "<cell>", line 1 for base in seq ^ SyntaxError: expected ':'</pre>	<pre>Traceback ----> 1 codon_table["TGA"] KeyError: 'TGA'</pre>	<pre>2</pre> ✓
loud — you find out at once	loud — you find out at once	this one is wrong correct answer is 3

Two look scary. The dangerous one looks fine.

Defect 1 · The off-by-one

- Does not raise anything
- Silently drops exactly one item — usually the first or the last
- On a 12,000-row table, losing one row changes almost nothing you would notice
- We already met this last week, in coordinates. It is the same error wearing different clothes.

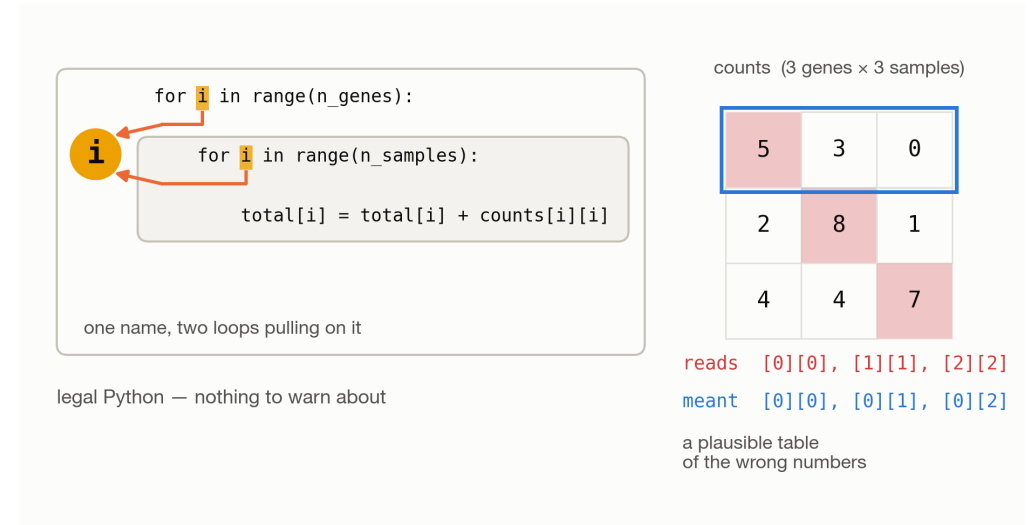


`range(1, len(x))` never examines `x[0]`. Nothing raises.

```
# intended: look at every base
for i in range(1, len(seq)):
    ...
# range(1, n) starts at 1 — the first base is never examined
# range(n)     starts at 0 — correct
```

Defect 2 · The reused variable

- The inner loop overwrites the outer loop's counter
- Nothing raises. Every index is valid. Every value is a number.
- The result is a coherent, plausible, entirely meaningless table
- Classic in nested loops, and classic in generated code where two blocks were stitched together

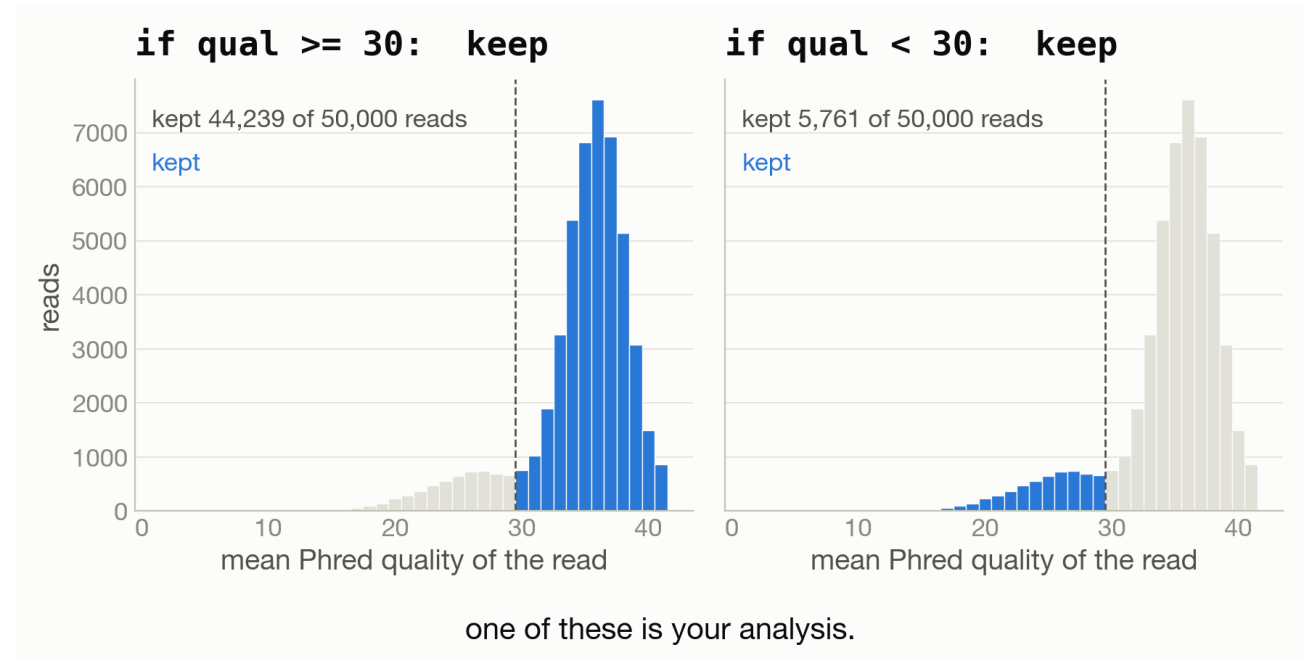


One name, two loops: the table reads its own diagonal.

```
for i in range(n_genes):  
    for i in range(n_samples):      # <-- same name  
        total[i] = total[i] + counts[i][i]
```


Defect 3 · The inverted condition

- Should be $\geq$. Is $<$.
- Runs perfectly. Produces a filtered list. The list is exactly wrong.
- Downstream, everything still works — on the wrong reads
- Also appears as not in the wrong place, or and where or was meant



one of these is your analysis.

```
# intended: keep high-quality reads
if qual < 30:
    keep.append(read)
```

None of the three raise anything

- Off-by-one · reused variable · inverted condition
- All three run
- All three return a plausible number
- All three change the answer
- The only thing that catches them is an input whose answer you already know



off-by-one

ran · returned a number



reused variable

ran · returned a number



inverted condition

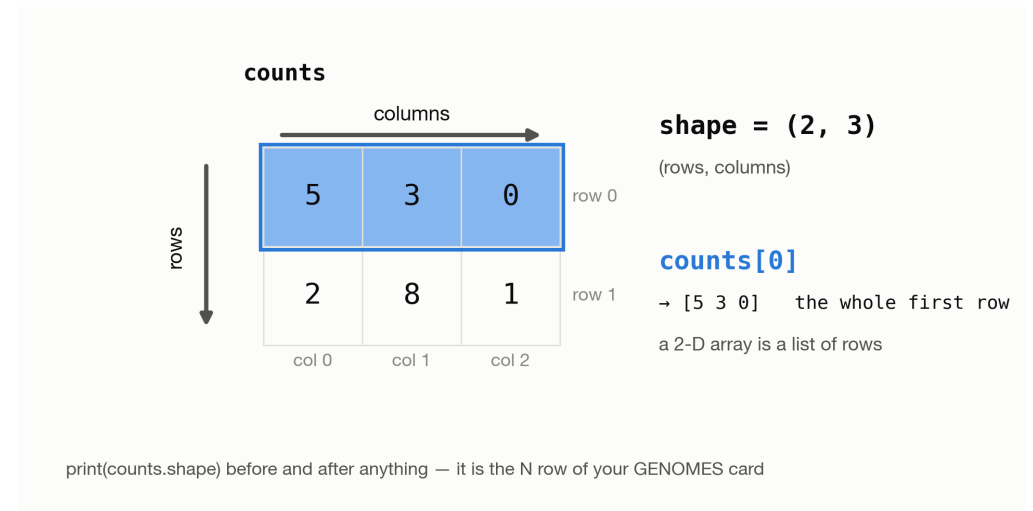
ran · returned a number

all three passed

all three passed

One array, two axes — the numpy floor

- An array (dizi) is a table with rows and columns
- `a[i]` gives you a whole row, not a single item
- `a.shape` prints (rows, columns). Print it before and after anything.
- `a.shape` is your N row on the GENOMES card, for free

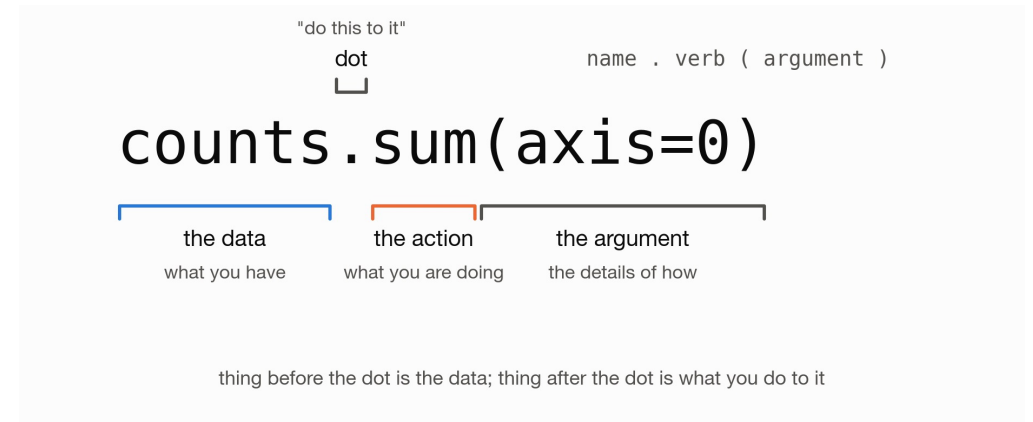


```
import numpy as np
counts = np.array([[5, 3, 0],
                  [2, 8, 1]])
print(counts.shape)    # (2, 3) -> 2 rows, 3 columns
print(counts[0])      # [5 3 0] -> the first ROW
```

counts[0] is a whole row; .shape is (rows, columns).

name.verb(argument) — the syntax nobody explains

- Thing before the dot = the data
- Thing after the dot = what you are doing to it
- The brackets hold the arguments — the details of how
- That is the whole of it. It is called a method call (metot çağrısı).



name . verb (argument)

<code>seq.count("G")</code>	<code># data: seq</code>	<code>• action: count</code>
<code>counts.sum()</code>	<code># data: counts</code>	<code>• action: sum</code>
<code>df.head(5)</code>	<code># data: df</code>	<code>• action: show me the first 5</code>
<code>"ATG".lower()</code>	<code># data: "ATG"</code>	<code>• action: lowercase it</code>

What we are NOT doing today

- We are not doing axis=0 versus axis=1 today
- A table has two directions you can collapse
- That idea arrives in Week 7, where the first check that needs it lives
- Today you own: `array · row · .shape · name.verb()`
- That is enough to read Friday's function 3, and enough for Weeks 4 and 6

The same 3×2 board (rate = count $\div$ kb). The axis you name is the one that disappears.

M.sum(axis=0)

	s1	s2
A	100	100
B	100	100
C	80	180

↓ ↓

280	380
-----	-----

[280, 380]

down the rows → 2 numbers, one per SAMPLE
(axis 0, the gene axis, disappears)

M.sum(axis=1)

	s1	s2
A	100	100
B	100	100
C	80	180

→ →

200
200
260

→ [200, 200, 260]

across the columns → 3 numbers, one per GENE
(axis 1, the sample axis, disappears)

not today — this arrives in Week 7, with the first check that needs it

Not today: axis arrives in Week 7, with the first check that needs it.

How a model reads code — and where that breaks

- Give a model a function with a planted defect and ask what it does
- It answers fluently, well-organised, and describes what the code was clearly intended to do
- It does not notice the defect
- Because it is pattern-matching against thousands of correct versions of this function
- A model's explanation is evidence about how the code looks, not about what it does

Friday's lab, and what is due

- Four short functions. Read, predict in writing, run, compare, state the purpose.
- Function 3 is array-shaped and it is correct — reporting a defect in it means you guessed instead of reading
- Function 4 has exactly one seeded defect from today's three families
- You compute the answer by hand for a five-element input, then run it
- A.finish() at minute 30. Everything after is optional and at home.
- Due Sunday 23:59. GENOMES rows E and N required; M still n/a until W6.

```
[1] import bib_colab as bib
A = bib.start("L03")

BIB L03 – Trace it, predict it, break it (100 points)
Questions to answer: Q1, Q2, Q3

Next: fill in the A.whoami(...) cell below.
```

```
[2] A.whoami(name="Ada Tiryaki", student_no="05220000895", section="EN")

L03 – Trace it, predict it, break it
Ada Tiryaki | 05220000895 | section EN
your dataset seed: 1386981810

This seed is derived from your student number. Your data is not the
same as anyone else's, so a borrowed answer will be the wrong answer.
```

```
[3] rng, params = A.dataset()

Your scenario:
seq_id          seq10
hand_seq        ATTTA
count_table     array, shape (3, 4)
```

md PREDICTION (write this before you run the cell below):
count_purines("ATGGC") → I predict it returns ____

```
[4] count_purines("ATGGC")
```

the prediction goes HERE, above the code cell

run the code cell only after the prediction is written

cells 1–3: identical to Weeks 1 and 2 — the output is printed by bib_colab.py itself

The prediction goes in the markdown cell above the code cell.

Sources on this slide set

- Python 3 tutorial (functions, lists, dicts) — <https://docs.python.org/3/tutorial/>
- NumPy absolute beginner's guide — https://numpy.org/doc/stable/user/absolute_beginners.html
- Python 2 sunset (why every old example in the old decks is a syntax error) — <https://www.python.org/doc/sunset-python-2/>
- PEP 8, on why indentation and naming are not cosmetic — <https://peps.python.org/pep-0008/>
- BioCoder benchmark lineage, Bioinformatics 2024;40(Suppl 1):i266 — https://academic.oup.com/bioinformatics/article/40/Supplement_1/i266/7700865
- Teufel & Liberles, JMBE 2026, doi:10.1128/jmbe.00331-25 — reasoning-first: commit to your own answer before seeing the model's